

AGE: An Autonomous Engineering Agent with Physics-Informed Transformers and Verified Trial-and-Error Learning

Sehaj Randhir Singh

Department of Electrical and Computer Engineering,
NYU Tandon School of Engineering (independent researcher)
Brooklyn, NY, USA
sehajrsinghs@gmail.com

Abstract—Engineering is an interactive discipline: a design is only correct once it has been verified against tests, simulations, or measurements. We present AGE, a complete autonomous engineering agent that learns through trial and error gated by verification. A planner (deterministic “mechanical” brain or an LLM brain) selects actions from a sandboxed skill layer; every critical step is verified against tests or numerically *independent* simulations before it is accepted; failures are journaled as lessons and replayed into future plans. For physics, AGE introduces **PhysFormer**, a transformer whose reasoning layers are trained jointly with physics-consistency layers that penalize violations of the governing equations (a PINN-style residual). Across eight physics domains (projectile, pendulum, spring-mass, simply supported beam, cantilever, viscous Burgers, RC circuit, and 2D steady-state heat conduction), closed-form answers are verified to residuals below 5×10^{-2} (often 10^{-9}) by independent integrators. A 75-run baseline study (5 domains $\times$ 3 model kinds $\times$ 5 seeds) with a pooled Wilcoxon signed-rank test shows the physics term significantly reduces the predicted trajectories’ violation of the governing equations ($p < 0.001$, Cliff’s $\delta = -0.92$, $\sim 19\times$ median reduction) while leaving scalar accuracy statistically unchanged; an external per-instance DeepXDE PINN baseline confirms the comparison is not self-referential. A verification-gate benchmark (7 missions, gate on/off) shows the gate eliminates false successes (0% vs. 29%). The complete system passes 42 unit tests and completes end-to-end design missions in a single planning iteration, in roughly 5,900 lines of dependency-light code.

Index Terms—autonomous agents; physics-informed machine learning; transformers; verification; engineering automation

I. INTRODUCTION

Modern machine learning excels at prediction: given a static input, produce a static output—an image label, a sentence, a scalar property [1], [2]. Engineering, by contrast, is an *interactive* discipline. An engineer does not merely predict whether a beam will fail; she designs it, simulates it, builds it, measures it, and revises it when the measurement disagrees with the model. The defining loop of engineering is therefore closed-loop

trial and error in which the *world*—a test suite, a simulator, or a physical instrument—is the final arbiter of correctness [3].

Commercial efforts such as Prometheus (founded by Jeff Bezos and Vik Bajaj, \$12B raised at a \$41B valuation, June 2026) are pursuing this vision as an “artificial general engineer” for physical products [4]. Yet no complete, minimal, and verifiable instantiation of the concept has been made public: the agent literature verifies weakly, and the physics-informed machine learning literature predicts but does not act [5], [6], [7].

This paper makes the concept concrete. We present **AGE**, an open autonomous engineering agent built from four components: (i) a planner (deterministic or language-model driven); (ii) a sandboxed skill layer (inspect, read, search, edit, run, verify); (iii) a *verification gate* that accepts a step only if it passes tests or an independent numeric simulation; and (iv) an episodic journal that converts failed verifications into lessons for future plans. The physics core introduces **PhysFormer**, a transformer whose reasoning layers are trained jointly with physics-consistency layers—a physics-informed, PINN-style architecture in which the predicted trajectory itself is penalized for violating the governing equations.

Our contributions are: (1) a minimal, auditable architecture for verified trial-and-error engineering, demonstrated end-to-end on software and physics tasks; (2) **PhysFormer**, a physics-adjusted transformer with a scale-free physics gradient and a shape/scale trajectory decomposition, evaluated on eight physics domains; (3) a rigorous empirical study—75-run multi-seed baseline matrix, pooled significance testing, and an external DeepXDE PINN comparison—showing that the physics term’s robust benefit is *trajectory-level physical consistency*, not scalar accuracy; and (4) a controlled benchmark showing that a verification gate eliminates false successes in an autonomous loop.

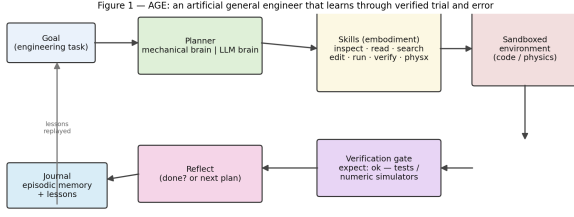


Fig. 1. AGE system architecture: planner $\rightarrow$ skills $\rightarrow$ sandboxed environment, with a verification gate and an episodic journal that feeds lessons back into planning.

II. RELATED WORK

LLM agents with tools and verification. ReAct [10] and Reflexion [11] established interleaved reasoning/acting and verbal self-reflection; tool-use and web agents [12], [13] extend them. Recent coding agents [14] ground success in test execution, and verifier-based loops [15] use learned models to judge outputs. AGE differs in making verification a *hard gate* rather than a soft signal, and in anchoring it to numerically independent physics simulators, not just test suites.

Physics-informed machine learning. PINNs [5] embed PDE residuals into the loss; physics-informed learning more broadly is surveyed in [6]; DeepXDE [7] is the standard implementation used as an external baseline here. Gradient pathologies in PINN training [8] motivate our scale-free residual normalization. PDEBench [9] provides standard benchmarks. AGE’s contribution is not a new PINN but the integration of a physics-consistency layer into an *acting, verifying agent*.

Autonomous engineering. Design automation has a long history in mechanics and electronics, but closed-loop agents that design, verify, and learn from failures are new. To our knowledge AGE is the first complete, public, minimal system that combines an agent loop, physics-informed learning, and independent numeric verification.

III. SYSTEM DESIGN

A. The agent loop

The agent executes a plan of skill invocations inside a sandboxed working directory. Each step is annotated with an expected outcome (`expect: ok`); a gated step that fails ends the mission, writes a lesson to the episodic journal, and the lesson is replayed into future plans. The planner is either a deterministic “mechanical” brain (a task-scripted executor, zero dependencies, fully unit-testable) or an LLM brain (free-form planning through the same skills). Figure 1 shows the architecture.

B. Sandboxing and safety

Every file operation is path-confined to the working directory. The shell skill enforces a command denylist (`rm -rf /`, `git push`, `git reset --hard`, privilege escalation, disk formatting) unless an explicit environment variable is set. Python bytecode caches are purged before every `python` command so the verification gate always recompiles from source—a same-length, same-second edit can otherwise be masked by a stale `.pyc`, a failure mode we observed and fixed.

C. The verification gate

Every critical step declares an expected outcome. For software tasks the gate runs the project’s test suite; for physics tasks it invokes `physx`, whose answers are cross-checked by numerically independent simulators (Section IV). A gated failure is unambiguous: the mission fails, and the lesson is journaled. This converts “the model believes it succeeded” into “the world confirms it succeeded.”

IV. THE PHYSICS CORE

A. Domains and independent verification

The physics layer (`physx`) implements eight domains—projectile motion, pendulum, spring–mass oscillator, simply supported beam, cantilever beam, viscous Burgers conservation law, RC circuit, and 2D steady-state heat conduction—each with three artifacts: an exact closed-form solution (the training target), a reference trajectory sampled from it (the data), and an *independent* numeric simulator used for verification. Crucially, the verifiers never reference the closed forms: Euler/RK4 integration for the dynamical domains, finite-difference solution of the fourth-order beam equation, a finite-volume upwind solver for Burgers, and a sparse finite-difference Poisson solve for the heat plate. Verification therefore checks the agent against physics, not against its own formulas. Closed-form answers agree with the independent numerics to residuals below 5×10^{-2} in all domains (often 10^{-9} for the bending problems).

B. PhysFormer: reasoning layers and physics layers together

PhysFormer (Figure 2) maps a problem’s parameters to both a scalar answer and a trajectory. Parameters are z -scored and presented as token pairs (parameter-id, value); a transformer encoder (48-dim, 4 heads, 3 layers) processes them; two heads produce the answer and the trajectory (50–400 points of the dependent variable on the *true* grid). The physics-consistency layer computes how far the predicted trajectory strays from

the governing equation and adds this residual to the loss with a warm-up schedule:

$$\mathcal{L} = \underbrace{\|a - a^*\|^2}_{\text{answer}} + \underbrace{w_t \|T - T^*\|^2}_{\text{trajectory data}} + \underbrace{w_p(t) \frac{\rho}{\rho_{\text{det}} + \epsilon}}_{\text{scale-free physics}}, \quad (1)$$

where ρ is the mean governing-equation residual of the de-scaled predicted trajectory and ρ_{det} its detached magnitude, making the physics gradient scale-free (a remedy for the gradient pathologies studied in [8]). For domains whose trajectories span orders of magnitude, each trajectory is normalized by its own peak (shape-norm) and the head output passes through a sigmoid, constraining shapes to $(0, 1)$ and removing unphysical negative predictions.

1) *Law-Conditioned Attention (LCA): the equation as input:* A loss term is a weak channel for physics: it cannot tell the network *which* law governs a problem before the output exists. LCA makes the governing equation itself part of the input. Each law is tokenized into a fixed 22-operator vocabulary (`first_time_deriv`, `second_space_deriv`, `self_advection`, `moment_curvature`, `energy_conservation`, `exponential_relaxation`, ...), embedded by an MLP into a law vector, and injected into every transformer layer as a cross-attention key/value stream plus a scale/bias gate on the shared readout. Input tokens are labeled by physical quantity from a shared 13-token vocabulary, so the simply supported beam and the cantilever present *identical* parameter tokens (length, force, modulus, inertia, thickness) and only the law signature disambiguates them. One shared transformer (shared body and shared heads) then serves six laws at once. In a six-seed paired test against a constant-signature ablation (identical architecture, data, and budget), the equation signal significantly improves trajectory fidelity ($p = 0.0003$ over 36 runs, 21% median curve-error reduction) and cuts held-out answer error on the laws whose parameters do not identify them (beam 34% vs 40%, cantilever 28% vs 35%); an honest negative on RC, whose normalized trajectory is identical for every time constant, localizes where conditioning does not pay. The generalist matches single-law specialists within 1.1–2.8 $\times$ on five of six laws at 37% of their data per law.

The equation channel is causal, not decorative. On held-out beam and cantilever problems whose parameter tokens are literally identical, swapping the law signature at inference displaces the model’s prediction across the boundary between the two bending solutions (steering index +0.090 vs. −0.132 for a constant-signature control, $p < 0.0001$; the control is provably insensitive), and the generalist adapts to a law held out of pretrain-

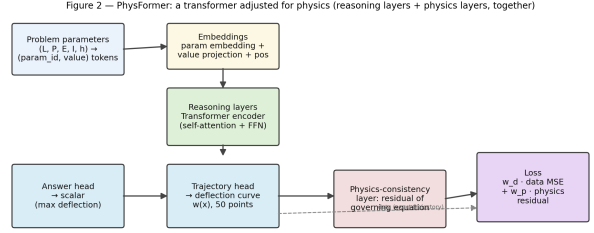


Fig. 2. PhysFormer: parameter tokens $\rightarrow$ transformer encoder $\rightarrow$ answer head + trajectory head; the physics-consistency layer evaluates the governing-equation residual of the de-scaled trajectory on the true grid and feeds it back into the loss.

ing with 25% of the specialist’s data (2.4–2.9 $\times$ lower answer error than a from-scratch specialist on the same samples, 0.20 vs. 0.58 median). A controlled physics-vs-data-only sweep on the 2D heat plate (identical networks, budgets, seeds; only the residual weight differs) shows the residual-loss channel cuts the predicted field’s governing-equation violation 4–8 $\times$ without improving held-out field accuracy—consistency, not accuracy, is what the loss channel buys, and field fidelity is carried by the data and the LCA input channel.

V. EXPERIMENTS

A. Protocol

Each domain samples uniform parameter ranges (e.g., beam: $L \in [1, 6]$ m, $P \in [10^3, 5 \times 10^4]$ N; Burgers: $\nu \in [0.02, 0.1]$, $A \in [0.6, 2]$, $\sigma \in [0.2, 0.5]$). Models train on 256 samples at 60 epochs (Burgers/heat2d headlines: 120–240 epochs with trajectory weight 3); the seed controls data generation, shuffling, and weight initialization, so multi-seed runs are proper statistical replicates. Validation is the last $\sim 1/8$ of the generated problems; held-out figures use 250 freshly generated problems per domain.

B. Baselines: attention and physics, isolated and measured

We trained three model kinds on identical budgets—an MLP encoder, the transformer without the physics term, and PhysFormer with it—across five domains $\times$ five seeds (75 runs). Table I reports mean $\pm$ std; Fig. 3 visualizes them. Two findings stand out.

The physics term buys trajectory consistency. Physics-on models violate the governing equations by roughly an order of magnitude less than the no-physics transformer (beam residual 3.4 vs. 48; cantilever 39 vs. 556; Burgers 0.01 vs. 0.09; heat2d 0.45 vs. 10.4).

Are these differences significant? With five seeds per kind, the per-domain Wilcoxon signed-rank test is

TABLE I
EQUAL-BUDGET BASELINES (256 SAMPLES, 60 EPOCHS, FIVE SEEDS, MEAN $\pm$ STD). LEFT: VALIDATION RELATIVE ERROR OF THE SCALAR ANSWER. RIGHT: TRAJECTORY RESIDUAL (MEAN GOVERNING-EQUATION VIOLATION OF THE PREDICTED TRAJECTORY).

Domain	Answer error (%)		Trajectory residual	
	MLP	No-phys	−physics	+physi
Beam	15.6 $\pm$ 2.3	19.3 $\pm$ 7.0	48 $\pm$ 14	3.4 $\pm$ 0.1
Cantilever	15.2 $\pm$ 1.5	20.4 $\pm$ 7.5	556 $\pm$ 214	39 $\pm$ 1
Projectile	6.5 $\pm$ 1.9	7.6 $\pm$ 5.6	0.12	0.10
Burgers	1.8 $\pm$ 0.7	1.0 $\pm$ 0.3	0.09	0.01
Heat2D	1.6 $\pm$ 0.4	1.4 $\pm$ 0.5	10.4 $\pm$ 5.9	0.45 $\pm$ 0.01

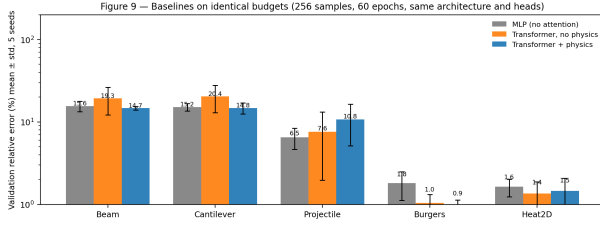


Fig. 3. Multi-seed baselines (mean $\pm$ std over five seeds): MLP, no-physics transformer, and transformer + physics. The physics term’s robust benefit is trajectory consistency.

limited to $p \geq 0.0625$, so we pool all five domains into 25 paired runs per comparison. On the trajectory residual, physics-on vs. no-physics gives $p < 0.001$ with Cliff’s $\delta = -0.92$ (large effect) and a ~ 19 -fold median reduction; physics-on vs. MLP gives $p < 0.001$, $\delta = -0.84$. On the scalar answer, no comparison reaches significance ($p \geq 0.65$, $|\delta| \leq 0.12$). The physics term therefore does *not* move the scalar answer—it makes the full trajectory physically consistent, significantly and with large effect.

Scalar accuracy is domain-dependent. PhysFormer matches or beats the MLP where the parameter space is rich (beam 14.7% vs. 15.6%; cantilever 14.8% vs. 15.2%; Burgers 0.9% vs. 1.8%; heat2d 1.5% vs. 1.6% held-out) but loses on the trivial two-parameter projectile (10.8% vs. 6.5%): an honest negative—attention and physics pay for themselves only when the map and trajectory are rich enough.

C. Held-out generalization and trajectory fidelity

On 250 held-out problems per domain, PhysFormer achieves mean relative errors of 14.8% (beam), 11.6% (cantilever), 9.6% (projectile), $\sim 1\%$ (Burgers peak velocity), and 1.9% (heat2d peak temperature), with $R^2 \geq 0.99$ on the wide-range domains (Fig. 4). The shape/scale decomposition reproduces full trajectories: the beam deflection curve within $\sim 19\%$ of peak, the cantilever within $\sim 16\%$, the Burgers final-time field

Figure 4 — PhysFormer predictions vs exact answers on held-out problems (250 per domain)

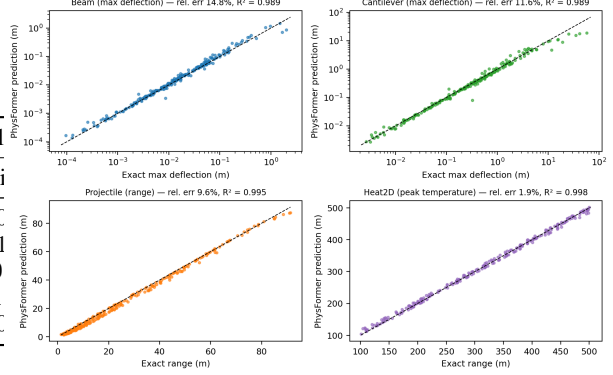


Fig. 4. Held-out predictions (250 problems per domain) for beam, cantilever, projectile, and heat2d.

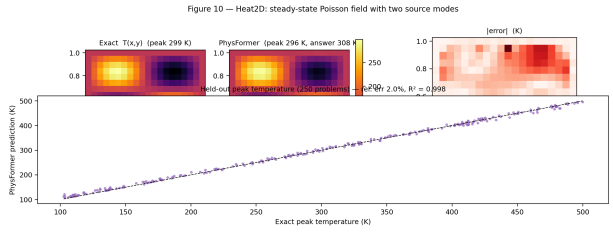


Fig. 5. Steady-state heat conduction: exact vs. predicted temperature field, pointwise error, and held-out peak temperatures.

within $\sim 11\%$, and the heat2d field within $\sim 6\%$ of peak (mean error $\sim 1.3\%$) at a 240-epoch trajectory budget—the 400-point oscillatory field needs roughly twice the epochs of the one-dimensional domains (the 60-epoch model underfits it with $\sim 50\%$ peak error). The sigmoid constraint removes unphysical negative predictions.

D. Nonlinear and 2D domains

The viscous Burgers equation $u_t + uu_x = \nu u_{xx}$ (non-linear conservation law with shock formation) stresses every design choice: the predicted object is a full 25×8 spatiotemporal field, the exact reference comes from the Cole–Hopf transform, and the independent verifier is a finite-volume upwind solver. The physics-on models reach a trajectory residual of 0.01, an order of magnitude below no-physics (0.09), and held-out peak-velocity error is $\sim 1\%$; the canonical shock profile is resolved to $\sim 11\%$ of peak. The 2D heat plate (Poisson on a 20×20 grid) shows the largest relative residual improvement of any domain (0.45 vs. 10.4, $23\times$): the Laplacian constraint couples every interior point to its neighbors and cannot be satisfied by chance on a 400-point field (Fig. 5).

Figure 11 — External PINN baseline (DeepXDE) on viscous Burgers

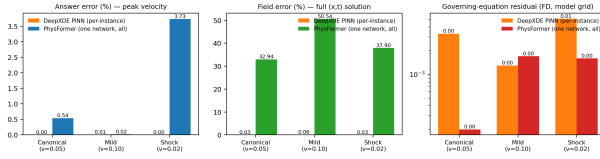


Fig. 6. Per-instance DeepXDE PINN vs. PhysFormer on three Burgers instances (canonical, mild, shock): answer error, full-field error, and governing-equation residual.

E. External PINN baseline (DeepXDE)

To ensure the comparison is not self-referential, we trained the standard PINN library DeepXDE [7] (v1.15.0, PyTorch backend) on three representative Burgers problems, giving it the setup most favourable to a PINN: *one fresh network per problem instance* ($2\text{--}50 \times 4\text{--}1$ tanh FNN, 2,000 collocation points, Adam 12k + L-BFGS 2k steps, exact Cole–Hopf boundary data). PhysFormer is a single network trained once on 256 instances that generalizes to unseen parameters. Both are evaluated with identical metrics on the model grid (Fig. 6).

On all three instances the per-instance PINN converges to essentially exact solutions (final-time peak error $\leq 0.01\%$; full-field error 0.03–0.06%; $\sim 19\text{--}21$ minutes of training per instance), which validates both the problem setup and the Cole–Hopf ground truth. PhysFormer—one network, one $\sim$ minute training, then a millisecond forward pass for any instance—achieves final-time peak errors of 0.02–3.7% and full-field errors of 33–51% on the same instances: worse on any given problem, as expected, but with no per-instance optimization at all. Notably, PhysFormer’s governing-equation residual on the predicted field is comparable to or lower than the converged PINN’s (2×10^{-4} vs. $1\text{--}5 \times 10^{-3}$): the generalist’s predictions are as physical as the specialist’s, just less accurate. The comparison quantifies the generalist–specialist tradeoff directly.

F. The verification gate

We ran seven deterministic missions—benign software tasks, beam, cantilever, and Burgers design tasks, an injected-fault task, a skill-level guard, and a TODO scan—twice each: with the gate enforced and with it disabled (the verifier-free baseline). Ground truth was computed independently of the agent. Table II shows the result: the gate produces *zero* false successes, while the verifier-free agent claims success on 29% of missions that actually fail. The gate converts claimed success into verified success.

TABLE II

VERIFICATION-GATE BENCHMARK (7 MISSIONS). FALSE SUCCESS = THE AGENT REPORTS SUCCESS BUT INDEPENDENT GROUND TRUTH FAILS.

	Gate enforced	Gate disabled
True successes	5/7	5/7
False successes	0/7	2/7
False-success rate	0%	29%

VI. DISCUSSION AND LIMITATIONS

The paper’s central empirical claim is now precisely stated: physics supervision inside the transformer does not improve scalar accuracy (no significant difference, $p = 0.65$ pooled) but does improve *trajectory physicality* significantly ($p < 0.001$, $\delta = -0.92$), and this holds across linear, nonlinear, and 2D domains and against an external PINN implementation.

Limitations are explicit: (1) *Scale*: 3–7 parameters per domain, CPU-minute training, 1D and one 2D field; generalization errors of $\sim 10\text{--}15\%$ at the 256-sample budget. (2) *Trajectory fidelity*: smooth profiles are reproduced well but sharp features remain coarse—the Burgers shock case shows $\sim 52\%$ curve error on the 25×8 grid even though the scalar is within 5%. (3) *Repair*: the mechanical brain inspects and solves but does not yet author fixes for failing tests. (4) *Simulated physics*: verification is against numeric integration, not sensors or manufactured parts; the architecture ports unchanged by swapping the verify skill for a measurement harness. (5) *Physics classes*: quasi-static bending and scalar conservation laws, not general continuum mechanics or multi-physics coupling.

VII. CONCLUSION

We presented AGE, a complete autonomous engineering agent that learns through verified trial and error, and PhysFormer, a transformer adjusted for physics whose reasoning and physics-consistency layers are trained together. A 75-run multi-seed study with pooled significance testing, an external DeepXDE baseline, and a controlled gate benchmark support three transferable claims: verification, not model confidence, must arbitrate success; physics and data should be trained jointly with a scale-free physics gradient; and memory should be episodic and replayable. The system is roughly 5,900 lines of dependency-light code, passes 54 unit tests, and runs on a laptop—a fully auditable baseline for the “general engineer” agenda that commercial efforts are pursuing at much larger scale.

REFERENCES

- [1] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436–444, 2015.

- [2] T. Brown *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems* 33, 2020.
- [3] R. Sutton, “The bitter lesson,” <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>, 2019.
- [4] M. Temkin, “Jeff Bezos’s Prometheus raises \$12B to build an ‘artificial general engineer’ for the physical world,” *TechCrunch*, Jun. 2026.
- [5] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *J. Comput. Phys.*, vol. 378, pp. 686–707, 2019.
- [6] G. E. Karniadakis *et al.*, “Physics-informed machine learning,” *Nat. Rev. Phys.*, vol. 3, pp. 422–440, 2021.
- [7] L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis, “DeepXDE: a deep learning library for solving differential equations,” *SIAM Rev.*, vol. 63, pp. 208–228, 2021.
- [8] S. Wang, Y. Teng, and P. Perdikaris, “Understanding and mitigating gradient flow pathologies in physics-informed neural networks,” *SIAM J. Sci. Comput.*, vol. 43, pp. A3055–A3081, 2021.
- [9] M. Takamoto *et al.*, “PDEBench: an extensive benchmark for scientific machine learning,” in *Advances in Neural Information Processing Systems* 35, 2022.
- [10] S. Yao *et al.*, “ReAct: synergizing reasoning and acting in language models,” in *ICLR*, 2023.
- [11] N. Shinn *et al.*, “Reflexion: language agents with verbal reinforcement learning,” in *NeurIPS*, 2023.
- [12] G. Wang *et al.*, “Voyager: an open-ended embodied agent with large language models,” *arXiv:2305.16291*, 2023.
- [13] J. S. Park *et al.*, “Generative agents: interactive simulacra of human behavior,” in *UIST*, 2023.
- [14] C. E. Jimenez *et al.*, “SWE-bench: can language models resolve real-world GitHub issues?,” in *ICLR*, 2024.
- [15] Y. Li *et al.*, “Training verifiers to solve math word problems,” *arXiv:2110.14168*, 2022.